

The Transformer Architecture: A Comprehensive Theoretical Summary

Source-Grounded Document Study

September 3, 2026

Abstract

This document provides a rigorous and complete theoretical summary of the Transformer architecture, originally introduced by Vaswani et al. in the landmark 2017 paper “*Attention Is All You Need*”. Grounded entirely on the course materials and technical documentation, this report details the historical background, core mathematical formulations, positional encoding, multi-head attention mechanisms, and architectural components of the Encoder-Decoder model.

1 Background and Context

Prior to 2017, the domain of sequence-to-sequence processing (including Natural Language Processing, Machine Translation, and Speech Recognition) was heavily dominated by **Recurrent Neural Networks (RNNs)** and their variants, such as **Long Short-Term Memory (LSTM)** and **Gated Recurrent Units (GRU)**.

These models process sequences token-by-token in a chronological order. While intuitive for language processing, this sequential nature introduced significant structural bottlenecks:

- **Inability to Parallelize:** Because the computation at timestep t depends on the output of timestep $t - 1$, the training process cannot be effectively parallelized across modern GPU/TPU architectures. Thus, training times scale linearly with the sequence length.
- **Vanishing and Exploding Gradients:** In extremely long sequences, backpropagating gradients through time (BPTT) leads to gradients vanishing or exploding, making network convergence highly unstable.
- **Failure in Long-Range Dependencies:** Although LSTMs/GRUs attempt to mitigate memory loss, information sequential transmission degrades over distance. For example, in the sentence “*The animal didn’t cross the street because it was tired,*” resolving the pronoun “*it*” to “*animal*” across long distance is mathematically challenging for RNNs as the information must be sequential.

The Transformer architecture represents a paradigm shift by moving from sequential reading to parallel viewing. It eliminates recurrence completely and relies **entirely on the Attention mechanism**, allowing every token to communicate directly with any other token in a single step.

2 Overall Encoder-Decoder Architecture

The Transformer is established as a highly modular foundation architecture, structured as an Encoder-Decoder pipeline:

2.1 The Encoder Block (Semantic Understanding)

The Encoder processes the input sequence and maps it into a series of context-aware latent representations. It consists of a stack of N identical layers (typically $N = 6$ in the baseline paper, and $N \geq 12$ in modern architectures). Each Encoder layer features:

1. **Multi-Head Self-Attention:** Allows every token to dynamically weigh and aggregate information from all other tokens.
2. **Position-wise Feed-Forward Network (FFN):** A fully-connected network applied to each token independently to build non-linear features.

2.2 The Decoder Block (Controlled Autoregressive Generation)

The Decoder is responsible for generating the target sequence autoregressively (one token at a time). It is also a stack of N identical layers, each containing three sublayers:

1. **Masked Multi-Head Self-Attention:** Prevents tokens from attending to subsequent (future) positions during training, enforcing causality.
2. **Encoder-Decoder Cross-Attention:** Enables the Decoder to attend directly to the latent representations produced by the Encoder, aligning source-to-target representations.
3. **Position-wise Feed-Forward Network.**

2.3 Modular Adaptations

Depending on the objective, these building blocks can be separated:

- **Encoder-Only Models** (e.g., BERT): Optimized for understanding and classification.
- **Decoder-Only Models** (e.g., GPT series): Specialized in autoregressive text generation.
- **Full Encoder-Decoder Models** (e.g., T5): Ideal for sequence transformation tasks like translation and summarization.

3 Tokenization and Embedding

Raw text must be converted into numerical vector spaces before processing:

- **Tokenization:** Splits raw text into token IDs. The primary strategies are:
 - *Word-level:* Large vocabulary size, suffers from out-of-vocabulary (OOV) issues.
 - *Subword-level* (e.g., BPE, WordPiece, Unigram): Divides rare or compound words into sub-word tokens (e.g., "Transformer is powerful" $\rightarrow$ ["Transform", "er", "is", "power", "ful"]). This is the most common approach.

- *Byte-level*: Processes raw bytes directly, bypassing traditional word vocabularies for massive multilingual scale.

- **Token Embedding**: Each token ID is mapped to a continuous semantic vector:

$$\mathbf{x}_i \in \mathbb{R}^{d_{\text{model}}}$$

where d_{model} represents the embedding dimension (e.g., $d_{\text{model}} = 512$ for the baseline, 768 for BERT-base, and > 1024 for large models). While embedding vectors encode semantic relationships, they contain zero information about sequential order or absolute positions.

4 Positional Encoding

Since the Transformer processes all tokens in parallel, it is naturally permutation-invariant. To preserve sequential order, a position vector $\mathbf{PE}_i$ is directly added to the token embedding $\mathbf{x}_i$:

$$\mathbf{z}_i = \mathbf{x}_i + \mathbf{PE}_i$$

where $\mathbf{z}_i$ is the final contextual embedding vector fed into the encoder.

4.1 Sinusoidal Formulations

The standard Transformer uses fixed sine and cosine functions of varying frequencies to compute positional encodings:

$$\mathbf{PE}_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right) \quad (1)$$

$$\mathbf{PE}_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right) \quad (2)$$

where:

- pos is the position index of the token in the sequence $(0, 1, 2, \dots)$.
- i is the dimension index within the embedding vector.
- d_{model} is the total embedding dimension of the model.
- Even indices of the vector utilize the sine function, and odd indices utilize the cosine function.

4.2 Key Design Advantages

- **Extrapolation**: The continuous periodic nature of sinusoids allows the model to generalize to sequence lengths longer than those observed during training.
- **Relative Distance representation**: Through trigonometric identity transformations, the model can easily learn linear transformations to project relative positional offsets.
- **Zero Parameters**: It adds no additional learnable parameters, preventing overfitting.

5 The Self-Attention Engine

The core engine of the Transformer is the self-attention mechanism, which enables dynamic context-sensitive representation learning.

5.1 Query, Key, and Value Projections

For an input embedding matrix X , three distinct continuous spaces are generated using learned projection matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (3)$$

where:

- **Query (Q):** Represents the query vector (“What information am I looking for?”).
- **Key (K):** Represents the key vector (“What information do I contain to match others?”).
- **Value (V):** Represents the content value (“The actual information I will provide once matched”).
- The learnable weight matrices are defined as:

$$W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

where d_k is the projection dimension (typically $d_k = d_{\text{model}}/h$, where h is the number of heads).

5.2 Scaled Dot-Product Attention

The mathematical formulation of the Scaled Dot-Product Attention is:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (4)$$

Where:

- $QK^\top$ computes the pairwise similarity scores (or attention logits) between all tokens in the sequence.
- $\sqrt{d_k}$ is the scaling factor. In high dimensions, dot-product magnitudes scale rapidly. If unscaled, values enter regions where the softmax function saturates with extremely small gradients, impeding backpropagation. Dividing by $\sqrt{d_k}$ stabilizes gradients during training.
- $\text{softmax}(\cdot)$ normalizes similarities into a probability distribution over the sequence.
- Multiplying by V computes the final weighted sum representation.

6 Multi-Head Attention (MHA)

A single attention head is limited to focusing on one type of context relation at a time. To capture multiple linguistic, structural, syntactic, and semantic relationships simultaneously, Multi-Head Attention projects Q, K , and V into h subspaces and performs attention in parallel.

6.1 Formulation

Each head i computes its own attention output:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{Attention}(XW_{Q,i}, XW_{K,i}, XW_{V,i}) \quad (5)$$

These outputs are then concatenated and linearly projected back to the original model space:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (6)$$

where:

- h represents the number of attention heads.
- $W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is the learned output projection matrix that aggregates the representations from all parallel heads.

7 Position-wise Feed-Forward Network (FFN)

Following the multi-head attention block, the representation of each token passes through a fully connected Feed-Forward Network individually and identically:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (7)$$

where:

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ are learned weights, and b_1, b_2 are biases.
- d_{ff} is the intermediate dimension, typically set to $4 \times d_{\text{model}}$ (e.g., $d_{\text{ff}} = 2048$ for $d_{\text{model}} = 512$).
- **Role:** This sublayer acts as a local Multi-Layer Perceptron (MLP) at each sequence position, incorporating non-linear feature abstractions without mixing information across positions.

8 Residual Connections and Layer Normalization

To stabilize training and enable the depth of models to scale to hundreds of layers, a residual connection and layer normalization wrap each major sublayer:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (8)$$

where:

- **Residual Connection** ($x + \text{Sublayer}(x)$): Solves vanishing gradients by providing a shortcut for gradient flow.
- **LayerNorm:** Normalizes inputs across features, stabilizing activation distributions.

9 Architectural Advantages over RNNs

The following table summarizes the fundamental advantages of the Transformer architecture over classic sequential structures:

Table 1: Comparative Analysis: RNN vs. Transformer

Feature	RNN / LSTM	Transformer
Parallel Training	No (Strictly sequential)	Yes (Highly parallelized)
Long-Range Context	Limited (Degrades sequentially)	Excellent (Direct $O(1)$ pairwise attention)
Scalability	Difficult	Highly Scalable ($\geq$ Billions of params)
Suitability for LLMs	Poor	Excellent (Modern standard)
Complexity per Layer	$O(n \cdot d^2)$	$O(n^2 \cdot d)$

10 Conclusion

The Transformer represents a watershed moment in artificial intelligence. By completely decoupling semantic representation learning from temporal or sequential processing, it overcame the computational constraints of RNNs. Through the combined synergy of Multi-Head Self-Attention, Sinusoidal Positional Encoding, and residual-normalized feedforward blocks, the Transformer has established itself as the structural backbone of modern Large Language Models and Foundation AI.